

Company Booster

Module 3 — Task & Objective Management
Technical documentation

Module	3 — Task & Objective Management
Feature built	Task & Objective Management (feature 9 of the specification)
Stack	React 19, Node.js, Express 5, MongoDB Atlas, Mongoose 9
External API	Google Gemini (document analysis)
Database	MongoDB Atlas, database empployee_management
Pages	3 — Task orbit, Task detail, Add work
Source size	About 5,400 lines across backend and frontend

How to read this

Sections 1–4 explain what was built and why. Sections 5–8 are the technical detail: database, backend, external API and frontend. Section 9 covers running it, section 10 how to make common changes on the spot, and section 11 is an honest list of what it does not do. Section 12 answers the questions most likely to be asked.

1. Scope and responsibility

The wider product is an employee management system split into four modules. This document covers **Module 3, Task & Objective Management**, and specifically the Task & Objective feature. The other three modules (Human Resources, Project & Collaboration, Administration & Productivity) are the responsibility of other team members and are not described here.

What this module does

- Reads an uploaded project document and drafts a set of tasks from it, with a suggested department, owner, priority and hour estimate for each.
- Assigns tasks to employees.
- Tracks progress as a percentage, calculated rather than typed in.
- Supports subtasks (a checklist) on every task.
- Accepts file uploads against a task.
- Lets people ask questions on a task, which stay marked open until answered.
- Shows all of a department's work as a priority diagram.

What was deliberately left out

- **Login and registration.** Another team member is building authentication, so this module has no user accounts or sessions. Nothing here depends on that work, and it can be added without changing the data model.
- **Project rooms.** The specification allows assigning work to a room as well as a person. Rooms belong to Module 2 and do not exist yet, so tasks are assigned to individuals only. The task schema would need one extra field to support rooms later.
- **The other Module 3 features.** Employee Performance, Feedback & Evaluation and the Project Budget Tracker are part of Module 3 but were not in scope for this stage.

2. Technology used

The stack was fixed by the team: JavaScript throughout, React on the front, Node and Express on the back, MongoDB with Mongoose, deployed to Vercel. Every choice below sits inside that.

Layer	Choice	Version	Why
Frontend	React	19.2	Team standard. Component model suits three related screens.
Routing	react-router-dom	7.18	Client-side routing between the three pages.
Build	Vite	8.1	Fast dev server; proxies /api to Express so there is no CORS setup in development.
Styling	Plain CSS	—	Competition rules allow CSS libraries but forbid page builders. Hand-written CSS with custom properties keeps the bundle small and every rule explainable.
Backend	Express	5.2	Team standard. Small, well understood routing layer.
Database	MongoDB Atlas	cloud	Team standard. Free tier; no local install needed by teammates.

Layer	Choice	Version	Why
ORM	Mongoose	9.8	Schema validation and population, which plain the driver does not give.
Uploads	multer	2.2	Multipart parsing, configured to keep files in memory.
PDF reading	pdf-parse	2.4	Extracts text from PDF uploads.
Word reading	mammoth	1.12	Extracts text from .docx uploads.
External API	Google Gemini	2.5-flash	Free tier, allowed by the competition rules, and the API named in our own specification.

Rules compliance

The competition forbids CMS platforms, core PHP, Firebase, Supabase and Django. None are used. It requires each member to integrate one external API: this module integrates **Google Gemini**. It requires SQL, NoSQL or local storage: MongoDB Atlas is used, with real Mongoose schemas.

3. How the pieces fit together

The system is in two halves that talk over HTTP and JSON. The browser never touches the database; everything goes through the Express API.

```

BROWSER (React, port 5173)
  pages -> src/lib/api.js -> fetch('/api/...')
  |
  | Vite dev server proxies /api to port 4000
  v
SERVER (Express, port 4000)
  routes/ -> controller/ -> service/ -> model/ (Mongoose)
  | |
  | v
  | MongoDB Atlas (cloud)
  v
  service/gemini.js -> Google Gemini REST API

```

Why the backend has four layers

Layer	Folder	Responsibility	Lines
Routes	backend/routes	Maps a URL and method to a controller function. No logic.	40
Controllers	backend/controller	Validates input, chooses status codes, shapes the response. Knows about HTTP, not about the database.	268
Services	backend/service	All database queries and all business rules. Knows nothing about HTTP.	614
Models	backend/model	Mongoose schemas: field types, defaults, validation.	128

The separation matters for one practical reason: the whole module first ran on an in-memory array before the database was connected. Because only the service layer knew where data lived, moving to MongoDB changed that one layer. Controllers and routes were untouched.

4. What happens when a document is imported

This is the most involved path in the module, so it is worth following end to end.

- 1 The user opens **Add work** and switches to **From a document**, then drops in a PDF, Word file, Markdown, CSV or plain text.
- 2 The browser sends it as multipart form data to POST `/api/ai/analyse`.
- 3 **multer** parses the upload and holds the file in memory as a Buffer. Nothing is written to disk.
- 4 `document_parser.js` picks a reader by file extension: pdf-parse for PDF, mammoth for .docx, plain UTF-8 decoding for text formats. The result is trimmed to 20,000 characters.
- 5 The service loads the department list and the employee list, including each employee's skills.
- 6 If a Gemini key is configured, the text, the departments and the team roster go to Gemini with a response schema that forces valid JSON back. If no key is set, or the call fails, a built-in reader parses the document's own structure instead.
- 7 The draft is normalised: department names checked against the real list, priorities checked against the four allowed values, hours clamped between 1 and 80, and suggested emails resolved to real employee records.
- 8 The draft is returned to the browser. **Nothing has been saved.**

- 9 The user reviews the list, edits titles, changes departments, owners, priorities and hours, and unticks anything they do not want.
- 10 Pressing create sends the kept tasks to `POST /api/ai/create-tasks`, which inserts them in one operation with source: 'ai' recorded against each.

Design decision: nothing is created automatically

The system proposes; a person approves. A tool that silently created a dozen tasks from a half-understood document would be worse than no tool, because somebody would have to find and undo them. The review step is the feature, not an obstacle in front of it.

5. Database design

MongoDB Atlas, accessed through Mongoose. Six collections. The connection is cached in `config/db.js` so a serverless platform reusing a container does not open a new connection on every request.

Collections

Collection	Holds	Documents now
tasks	The work itself, with subtasks embedded	27
employees	People who can be assigned work	12
departments	Engineering, Design, Marketing, Human Resources	4
objectives	Goals that tasks can belong to	5
comments	Questions and comments on tasks	3
attachments	Uploaded files, stored as binary	0 until a file is uploaded

The task schema in full

Field	Type	Notes
title	String	Required.
description	String	Free text.
department	String, indexed	Which team owns it.
assignee	ObjectId → Employee	Null when unassigned.
objective	ObjectId → Objective	Null when the task stands alone.
priority	Enum, indexed	critical, high, medium, low.
status	Enum, indexed	todo, in_progress, review, done.
estimateHours	Number	Used for planet size and hour totals.
spentHours	Number	Reserved for time logging.
dueDate	Date	Drives the overdue calculation.
subtasks	Embedded array	Each has title, done, completedAt.
attachments	Array of ObjectId	References to the attachments collection.
source	Enum	manual or ai — records where the task came from.
aiReason	String	The sentence explaining why the AI proposed it.
createdAt / updatedAt	Date	Added automatically by Mongoose timestamps.

Design decision: progress is never stored

There is no progress field. It is calculated on every read by `progressPercent()` on the schema: 100 if the task is done, otherwise the share of ticked subtasks, and if there are no subtasks a value implied by the status. Storing a percentage would let it drift out of step with the checklist underneath it the moment somebody ticked a box. The same applies to *overdue* and *remainingHours*.

Relationships

```
Task -> Employee    many-to-one (assignee)
Task -> Objective    many-to-one (optional)
Task -> Attachment  one-to-many
Task <- Comment     one-to-many (comment holds the task id)
Subtasks are embedded inside the task document, not a collection.
```

Subtasks are embedded because they are never queried on their own and never shared between tasks; they are always read and written with their parent. Comments and attachments are separate collections because a task with many comments would otherwise grow without limit, and MongoDB caps a document at 16 MB.

Where uploaded files live

File bytes are stored in the attachments collection as a Buffer, not on the server's disk. This is deliberate: Vercel's filesystem is read-only, so writing uploads to a folder would work on a laptop and fail in production. Uploads are capped at 5 MB, well under MongoDB's document limit. The `data` field is marked `select: false`, so listing a task's files never drags the binary along with it.

6. The API

Fifteen endpoints, all returning JSON. Base URL `/api`.

Task endpoints

Method	Path	Purpose
GET	<code>/api/tasks/board</code>	Departments with counts, plus tasks for one department. Powers page 1.
GET	<code>/api/tasks/options</code>	Departments, employees, objectives and the allowed enum values, for the form dropdowns.
GET	<code>/api/tasks/:id</code>	One task with its employees list and comment thread. Powers page 2.
POST	<code>/api/tasks</code>	Create one task.
PATCH	<code>/api/tasks/:id</code>	Update any editable field.
DELETE	<code>/api/tasks/:id</code>	Delete a task, and its comments and attachments with it.
POST	<code>/api/tasks/:id/subtasks</code>	Add a checklist item.
PATCH	<code>/api/tasks/:id/subtasks/:subtaskId</code>	Tick or untick a checklist item.
POST	<code>/api/tasks/:id/comments</code>	Post a comment or a question.
POST	<code>/api/tasks/:id/attachments</code>	Upload a file (multipart).
GET	<code>/api/tasks/:id/attachments/:fileId</code>	Download a file.
DELETE	<code>/api/tasks/:id/attachments/:fileId</code>	Remove a file.

AI endpoints

Method	Path	Purpose
GET	<code>/api/ai/status</code>	Reports whether a Gemini key is configured and which file types are accepted.
POST	<code>/api/ai/analyse</code>	Reads a document and returns a draft. Saves nothing.
POST	<code>/api/ai/create-tasks</code>	Creates the tasks the user approved from that draft.

Rules the server enforces

- A task must have a title and a department.
- Priority and status must be one of the allowed values; anything else is rejected with 400.
- An estimate must be between 1 and 200 hours.
- Uploads are capped at 5 MB for attachments and 8 MB for documents; over that returns 413.
- Unsupported file types are rejected by name before any parsing is attempted.

Validation is on the server, not only in the form. A form can be bypassed; the endpoint cannot.

Two automatic status rules

These live in the service layer so every route that changes a subtask gets them:

- Ticking the first checklist item on a task that is still *to do* moves it to *in progress*. Without this the board and the task page would disagree about whether work had started.
- Ticking the last remaining item moves the task to *in review* rather than straight to done, because finishing the checklist is not the same as someone having checked the work.

7. The external API integration

The competition requires one external API integration per member. This module integrates **Google Gemini**, called directly over its REST endpoint with `fetch` rather than through an SDK, which keeps one dependency out of the project.

How the call is made

```
POST https://generativelanguage.googleapis.com/v1beta/models/
      gemini-2.5-flash:generateContent?key=API_KEY

body:
  systemInstruction the rules the model must follow
  contents          the document text, department list, team roster
  generationConfig
    temperature     0.3 (low, so results are repeatable)
    responseMimeType application/json
    responseSchema   the exact shape required back
```

The response schema is the important part. Gemini is told the precise JSON structure it must produce, so the reply is parsed directly instead of having prose pulled apart with string matching. A 45-second timeout is enforced with an `AbortController`.

The fallback, and why it exists

If no API key is set, or the call fails for any reason, a built-in reader in `fallback_planner.js` produces the same shape of result. It works on the document's own structure: headings, bullet lists, numbered lists, and prose sentences that begin with an action verb. It infers a skill set from keywords, matches that to the department whose people hold those skills, and estimates hours from the length and wording of each line.

Why build a fallback at all

A live demonstration should not depend on a network connection or an unexpired key. If Gemini is unreachable the feature still works, and the interface says plainly which engine produced the result rather than pretending an AI was involved. This is also an honest answer to the question "what happens when the API is down?"

A real defect this uncovered

Testing with a generated PDF showed the import producing nonsense: titles with line breaks through them and text that was clearly not a task. The cause was that a bullet character, `U+2022`, came back from PDF text extraction as a plain quote mark, `U+0022`, so the bullet pattern never matched and the parser fell back to splitting prose into sentences.

Two fixes followed. The bullet pattern now recognises the characters that PDF extraction actually produces, including `U+F0B7` which Word emits when exporting bulleted lists, and every title has its whitespace collapsed so a line break can never appear inside one. Six bullet variants were then tested and all parse correctly. The same brief now gives identical results as PDF, Word and Markdown.

8. The frontend

Three pages, deliberately. An earlier version of this module had eight, and it was harder to use, not easier — the same task could be reached four different ways and no screen was clearly the place to start.

Page	Route	What it is for
Task orbit	/	One department's open work drawn by priority, with a breakdown and a full task table under it.
Task detail	/tasks/:id	Everything about one task: description, checklist, files, questions, and the controls to change it.
Add work	/tasks/new	Two modes: create one task by hand, or import a document. Both live here so there is one place to add work.

The orbit diagram

The first page draws each department's tasks as a solar system. The rules are deliberately few, so the picture can be explained in one sentence: **distance from the centre and colour both mean priority, and nothing else.**

What you see	What it means
Ring position	Priority. Critical is the innermost ring, then high, medium, low.
Colour	The same priority. Colour and position never disagree.
Planet size	Estimated hours, clamped so one large task cannot swamp the picture.
White arc	Progress on that task.
Red halo	The task is past its due date.
Rotation speed	Critical rings turn faster than low ones. Motion is off by default.

An earlier version coloured planets by priority while positioning them by time remaining. Those are two different things, so a green planet and a yellow planet could share a ring and the diagram contradicted itself. Tying both to one dimension fixed it.

Design decision: the diagram is never the only view

A picture cannot tell you which three tasks are late or how the hours split. Underneath the orbit sits a breakdown — late, due within three days, open, unowned, a stage-by-stage count, and hours left split by priority — and a full sortable table of every task. The orbit is for the first glance; the table is what the work is actually done from.

How the frontend is organised

Path	Contents
src/lib/api.js	Every call to the server. One place for the base URL and error handling.
src/lib/format.js	Date, file size and label helpers, so dates read the same on every page.
src/styles/theme.css	Design tokens as CSS custom properties: the four brand colours and everything derived from them.
src/components/	solar_system (the diagram) and app_header (the top bar).
src/pages/	One folder per page, each with its .jsx and its .css beside it.

Colours are defined once in `theme.css` and referenced everywhere else, so the palette can be changed in one file. Motion is CSS keyframes and transitions only — no animation library — and the whole interface honours the operating system's reduce-motion setting.

9. Running the project

First time

```
backend:
  cd backend
  npm install
  copy .env.example to .env and fill in MONGODB_URI
  npm run seed    fills the database with starting data
  npm run dev     starts the API on port 4000
```

```
frontend (a second terminal):
  cd frontend
  npm install
  npm run dev    starts the site on port 5173
```

then open `http://localhost:5173`

Environment variables

Variable	Required	Purpose
MONGODB_URI	Yes	The Atlas connection string. The server exits if it is missing.
GEMINI_API_KEY	No	Enables Gemini. Without it the built-in reader is used instead.
GEMINI_MODEL	No	Defaults to gemini-2.5-flash.
PORT	No	Defaults to 4000.

The .env file is not committed

It is listed in `.gitignore`, and `.env.example` is committed in its place with the values blanked. A connection string contains a password; putting one in version control hands the database to anyone who can read the repository.

The demo files

The `demo/` folder holds a realistic client brief as Markdown, plus the same brief generated as a PDF and a Word document, so the import can be shown with the formats a client would really send. Running `node demo/make_demo_files.js` regenerates the PDF and Word versions after editing the Markdown.

10. Making common changes

Where to go if a change is asked for during the presentation.

Change asked for	Where to make it
Add a department	Add it to <code>DEPARTMENTS</code> in <code>backend/data/dummy_data.js</code> and re-seed. Every count, dropdown and tab picks it up automatically.
Add an employee	Add to <code>EMPLOYEES</code> in the same file and re-seed. Skills are inferred from the job title.
Add a fifth priority	<code>PRIORITIES</code> in <code>backend/model/task.js</code> , then <code>ORBITS</code> in <code>solar_system.jsx</code> for the new ring, then the colour variables in <code>theme.css</code> .
Change the colours	<code>theme.css</code> only. Every component reads the custom properties.
Change a status name	<code>STATUSES</code> in <code>backend/model/task.js</code> and <code>STATUS_LABELS</code> in <code>frontend/src/lib/format.js</code> .

Change asked for	Where to make it
Turn the orbit motion on by default	One line in task_orbit.jsx: the initial value of the animate state.
Change the upload size limit	backend/middleware/upload.js, the fileSize limits.
Change what the AI is told	The SYSTEM constant in backend/service/gemini.js.
Add a field to a task	The schema in model/task.js, then EDITABLE in service/task_service.js so it can be updated, then the form.

11. Known limitations

Stated plainly, because being asked about a weakness that has already been acknowledged is a much better position than being caught by it.

Limitation	Detail and what would fix it
No authentication	Anyone who can reach the API can change any task. Another member is building login; this module would then check a token on each request.
Comment authors are typed in, not verified	A comment records a name as text because there are no user accounts yet. It becomes a real reference once login exists.
The fallback reader misses cross-references	It reads each line separately, so a constraints paragraph saying "the login work is critical" does not raise the priority of a bullet further up the page. Gemini reads the whole document and does. With no key set, every task comes back medium.
The orbit does not scale past about 30 tasks	Rings are capped and the surplus is reported as a count. Beyond that the table underneath is the usable view, which is why it is there.
Scanned PDFs produce nothing	Text extraction needs real text. An image of a page would need OCR, which is not included.
No pagination	Every task for a department is sent at once. Fine at this size; a few hundred tasks per department would need paging.
Attachments are limited to 5 MB	They are stored inside MongoDB documents. Larger files would need object storage such as S3.
No automated test suite	Testing was done by exercising the endpoints and the interface directly. Unit tests would be the next thing to add.

12. Questions you are likely to be asked

Why MongoDB rather than a relational database?

It was the team's fixed choice, and it suits the shape of this data: a task with its checklist is one document, read and written together. Where relationships matter — a task's assignee, its objective — they are ObjectId references and Mongoose populates them, which is effectively a join. If the project needed heavy reporting across many joined tables, SQL would be the better answer.

Why are subtasks embedded but comments a separate collection?

Subtasks are always read with their parent task, are never queried alone, and there are only a handful. Comments can grow without limit and a MongoDB document is capped at 16 MB, so letting them accumulate inside the task would eventually break it.

Is the AI part real, or is it hard-coded?

It is a real HTTP call to Google Gemini with the document text and the team roster, using a response schema so the reply comes back as valid JSON. There is also a non-AI fallback that runs when no key is configured, and the interface states which of the two produced the result. It can be demonstrated either way.

What happens if the Gemini API is down or the key expires?

The call is wrapped in a try/catch with a 45-second timeout. On any failure the built-in reader runs instead and the response carries a notice explaining what happened. The feature degrades rather than breaking.

How is the progress percentage calculated?

It is not stored. On every read: 100 if the task is done, otherwise the proportion of ticked subtasks, and if there are no subtasks a value implied by the status. This means it can never contradict the checklist a user is looking at.

Where are uploaded files kept?

As binary inside the attachments collection in MongoDB, not on the server's disk. The deployment target is Vercel, whose filesystem is read-only, so a disk-based approach would work locally and fail in production.

Why only three pages?

An earlier version had eight and was harder to use. Related things were spread across separate screens, and the same task could be reached several ways with no obvious starting point. Document import was folded into the Add work page rather than given a page of its own.

Show me that the data is really in the database and not in memory.

Stop the server, start it again, and reload the page. The tasks, including any just created, are still there. This was verified during development: 32 tasks before a restart, 32 after.

What is the data in the demo — is it real?

It is written sample data, not an export from a real company. What makes it usable is that it is internally consistent: every figure on screen — progress, hours remaining, overdue counts, department totals — is calculated from those records at the moment of the request. No number is typed into the design.

Could this work with project rooms as the specification describes?

Yes, and it was left out only because rooms belong to Module 2 and do not exist yet. The task schema would take one more optional reference field, and the assignee logic would accept either an employee or a room.

What was the hardest problem you hit?

Reading PDFs reliably. Bullet characters do not survive text extraction intact — a bullet came back as a quote mark, which meant the parser could not see the list at all and produced nonsense. Fixing it required recognising the characters extraction actually emits, including the one Word uses, and normalising whitespace so line breaks from the printed page could not end up inside a task title.

One thing worth saying out loud

This module was built to be defended, not just demonstrated. Progress is calculated rather than claimed, validation is enforced on the server rather than only in the form, the AI proposes rather than decides, and the limitations above are stated rather than hidden.